IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

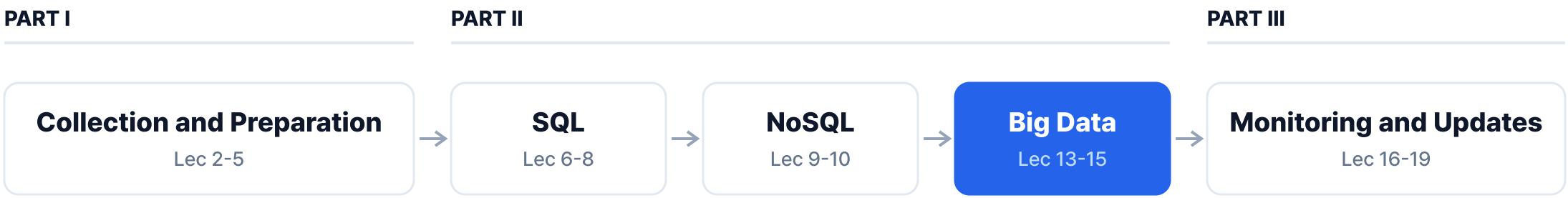
13.

Big Data I: Batch Processing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 13-15
Monitoring and Updates
Lec 16-19

Recap: MongoDB in One Slide

```
db.inventory.find(
    { tags: "red" },            // WHERE
    { _id: 0, instock: 0 } )    // SELECT
  .sort( { "dim.0": -1 } )      // ORDER BY
  .limit(2)                     // LIMIT
```

```
db.zips.aggregate( [
  { $group: { _id: "$state", totalPop: {
$sum: "$pop" } } },
  { $match: { totalPop: { $gte: 200000 } }
},
  { $sort:  { totalPop: -1 } }
] )   // GROUP BY, HAVING, ORDER BY as
stages
```

Document store: input = collections, output = collections. Retrieval is `find`, aggregation is a **pipeline of stages** in the order you write.

Outline: Big Data I

01	**Processing on Many Machines**	split a large job across machines
02	**How Spark Runs a Job**	understand when and where work runs
03	**Working with Spark**	query data and save the results

Beyond One Collection

- Data models:
  - Key-value
  - JSON
- We queried one MongoDB collection at a time
- Now: *parallel batch processing* over many machines

Why Distribute Batch Work

- Relational databases can also execute queries in parallel
- Large batch jobs may need custom record-processing functions
- MapReduce is a distributed programming model for such computation
- First, let’s study how data is stored in such systems

Distributed File System (DFS)

- For very large files: TBs, PBs
- Each file is partitioned into **chunks**, 64 MB in the original GFS design
- Replication factor and rack placement are configured for fault tolerance
- Implementations:
  - Google's DFS: GFS, proprietary
  - Hadoop's DFS: HDFS, open source

MapReduce

- Google: paper published 2004
- Free variant: Hadoop
- MapReduce: high-level programming model and implementation for large-scale parallel data processing

Case study: Google's Web Index

Google rewrote its production indexing system for web search on MapReduce

> 20 TB

raw crawled documents, input to the indexer

5 to 10

MapReduce jobs in a row build the index

3,800 → 700

lines of C++ for one indexing phase

MapReduce moved **distribution and recovery** code out of the indexing application

Typical Problems Solved by MapReduce

- Read a lot of data
- **Map**: extract something you care about from each record
- Shuffle and Sort
- **Reduce**: aggregate, summarize, filter, transform
- Write the results

Paradigm stays the same,
change map and reduce functions for different problems

Data Model

Files!

A file = a bag of `(key, value)` pairs

- A MapReduce program:
  - Input: a bag of `(inputkey, value)` pairs
  - Output: a bag of `(outputkey, value)` pairs
    - outputkey is optional

Step 1: the MAP Phase

User provides the **MAP**-function:

- Input: `(input key, value)`
- Output: bag of `(intermediate key, value)`

System applies the map function in parallel to all `(input key, value)` pairs in the input file

Step 2: the SHUFFLE Phase

This is completely internal to the system

- Input: Pairs of `(intermediate key, value)`
- Output: Pairs of `(intermediate key, bag of values with the same intermediate key)`

I.e., group all values with the same key

Step 3: the REDUCE Phase

User provides the **REDUCE** function:

- Input: `(intermediate key, bag of values)`
- Output: bag of output `(values)`

System groups all pairs with the same intermediate key, and passes the bag of values to the **REDUCE** function

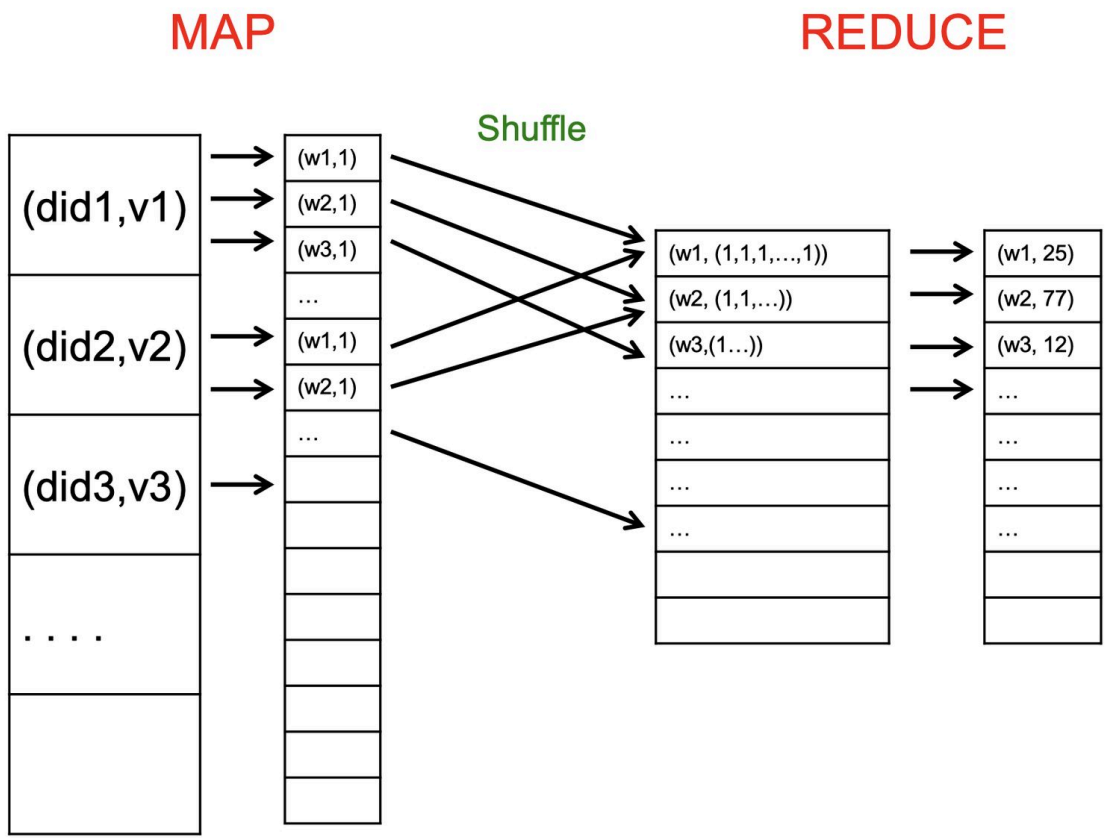
Example

- Counting the number of occurrences of each word in a large collection of documents
- Each Document
  - The key = document id (did)
  - The value = set of words (word)

```
map(String key, String value):
  // key: document name
  // value: document contents
  for each word w in value:
    emitIntermediate(w, "1");
```

```
reduce(String key, Iterator values):
  // key: a word
  // values: a list of counts
  int result = 0;
  for each v in values:
    result += ParseInt(v);
  emit(AsString(result));
```

Example: The Data Flow

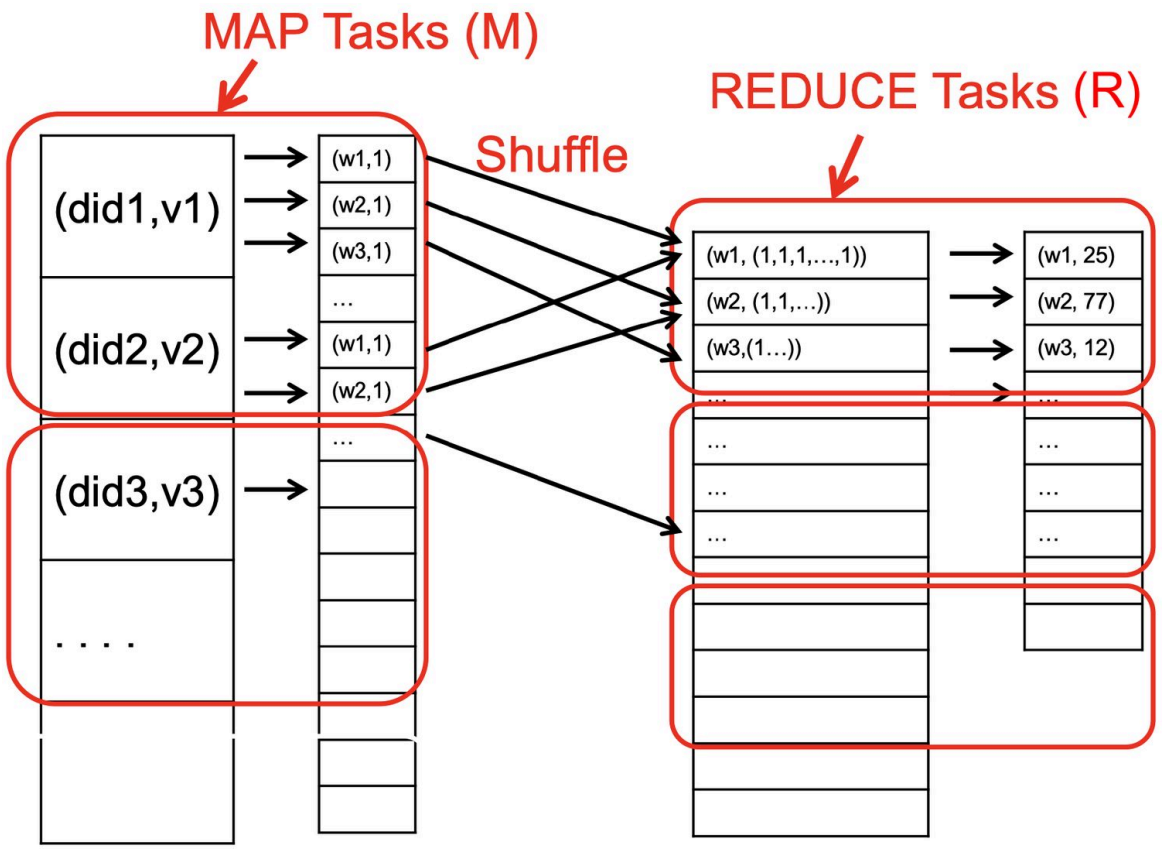


Map emits pairs, **shuffle groups them by key**, reduce folds each group

Workers

- A **worker** is a process that executes one task at a time
- This simplified model uses one task slot per worker; real runtimes configure concurrency

Map Tasks and Reduce Tasks



M map tasks and R reduce tasks, usually **many more than there are workers**

Fault Tolerance

- With independent server failures averaging one per year, 10,000 servers see a failure about every **53 minutes on average**
- MapReduce handles fault tolerance by writing intermediate files to disk:
  - Mappers write file to local disk
  - Lost map output is recomputed; failed reduce tasks restart on another worker

Demo: MapReduce Word Count

Reducer load is measured by input-value count, not elapsed time

1. Vary **R** from 1 to 4 and watch the largest load
2. Turn on the **hot key** and vary R again

Reducers 1 2 **3** 4 ☐ Hot key: `alarm` in every log ◀ Back Step ▶ Run all 3 · Reduce

Input splits

SPLIT 1
d1 pump leak valve pump
d2 motor heat pump

SPLIT 2
d3 valve motor belt
d4 belt pump heat valve

SPLIT 3
d5 motor valve leak
d6 heat belt motor pump motor

Map output *(word, 1)*

MAP 1
pump,1 R2 | leak,1 R1 | valve,1 R1 | pump,1 R2 | motor,1 R1 | heat,1 R3 | pump,1 R2

MAP 2
valve,1 R1 | motor,1 R1 | belt,1 R2 | belt,1 R2 | pump,1 R2 | heat,1 R3 | valve,1 R1

MAP 3
motor,1 R1 | valve,1 R1 | leak,1 R1 | heat,1 R3 | belt,1 R2 | motor,1 R1 | pump,1 R2 | motor,1 R1

Shuffle by `hash(word) mod R` → reduce *(sum)*

REDUCER 1
motor 5
valve 4
leak 2
11 values · largest load

REDUCER 2
pump 5
belt 3
8 values

REDUCER 3
heat 3
3 values

Largest load: **11 values** (even split: 8)

Answer: MapReduce Word Count

A hot key creates a load floor that additional reducers cannot remove

reducers R	max load, no hot key	max load, hot key	even split, hot key
1	22	46	46
2	13	33	23
3	11	27	16
4	8	29	12

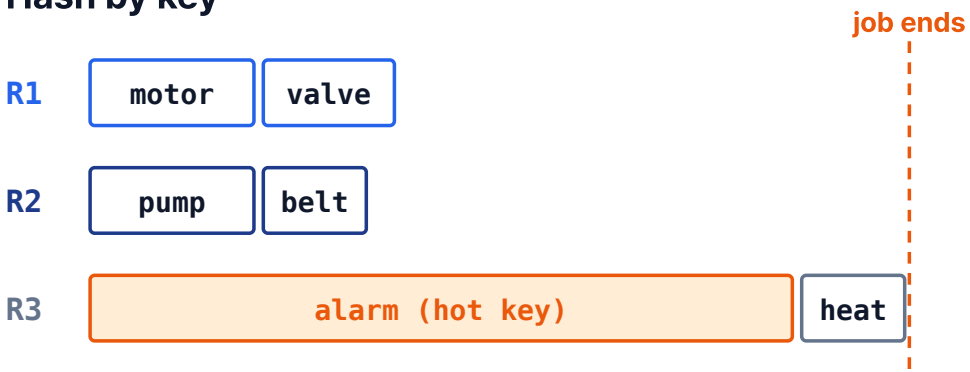
Hashing sends **all values of one key to one reducer**, so the reducer with `alarm` reads at least 24 values for any R

With the hot key, **R = 4 carries more load than R = 3**

Caveat: Data Skew

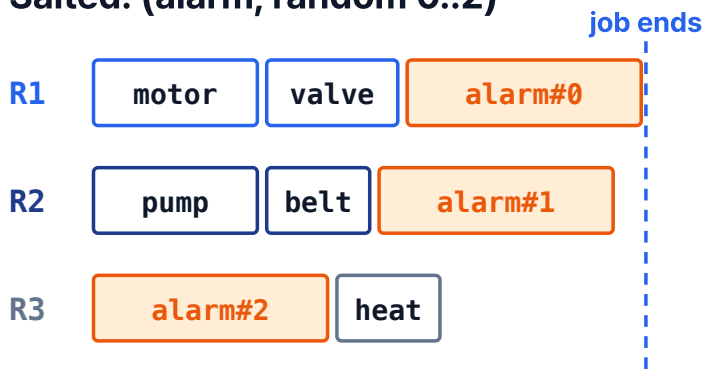
One hot key makes one task the bottleneck while the others sit idle

Hash by key



one reducer reads every alarm value

Salted: (alarm, random 0..2)



if the partitioner spreads them, a small pass sums the parts

Remedy: for counts, salt the hot key, count the parts, then sum the partial counts

Salting divides the hot key into **partial counts** before a final merge

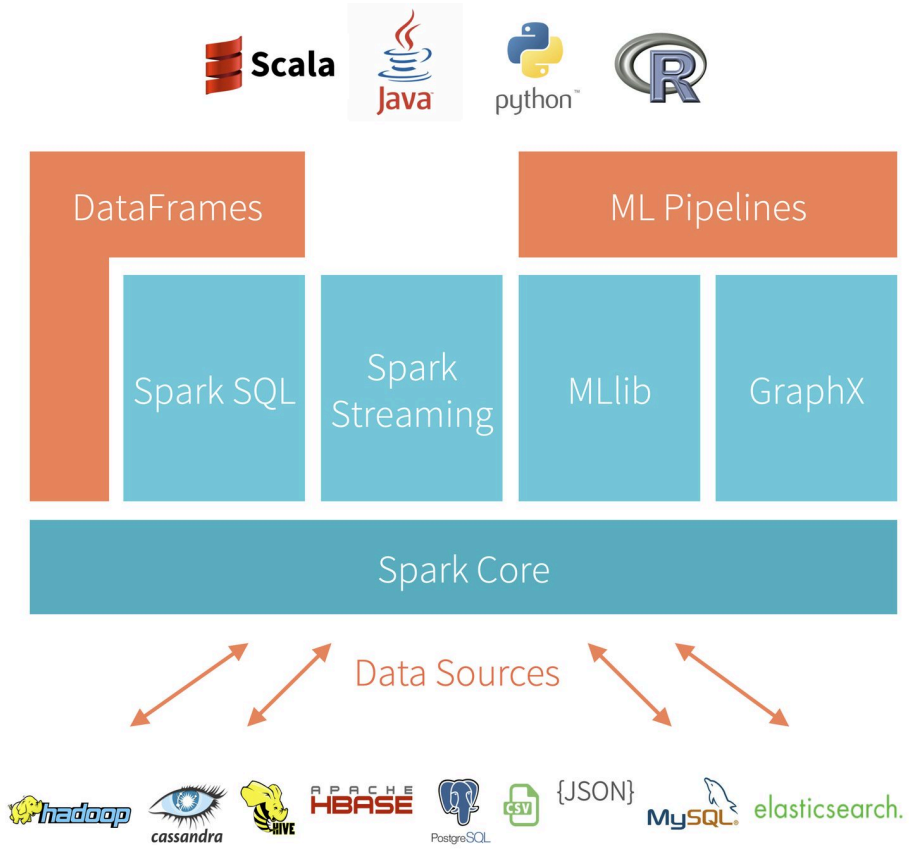
MapReduce Summary

- MapReduce offers a simple abstraction, and handles distribution + fault tolerance
- Speedup from dynamically allocating map and reduce tasks and to servers. Skew is possible!
- Write intermediate results to disk for fault tolerance but can be slow

Issues with MapReduce

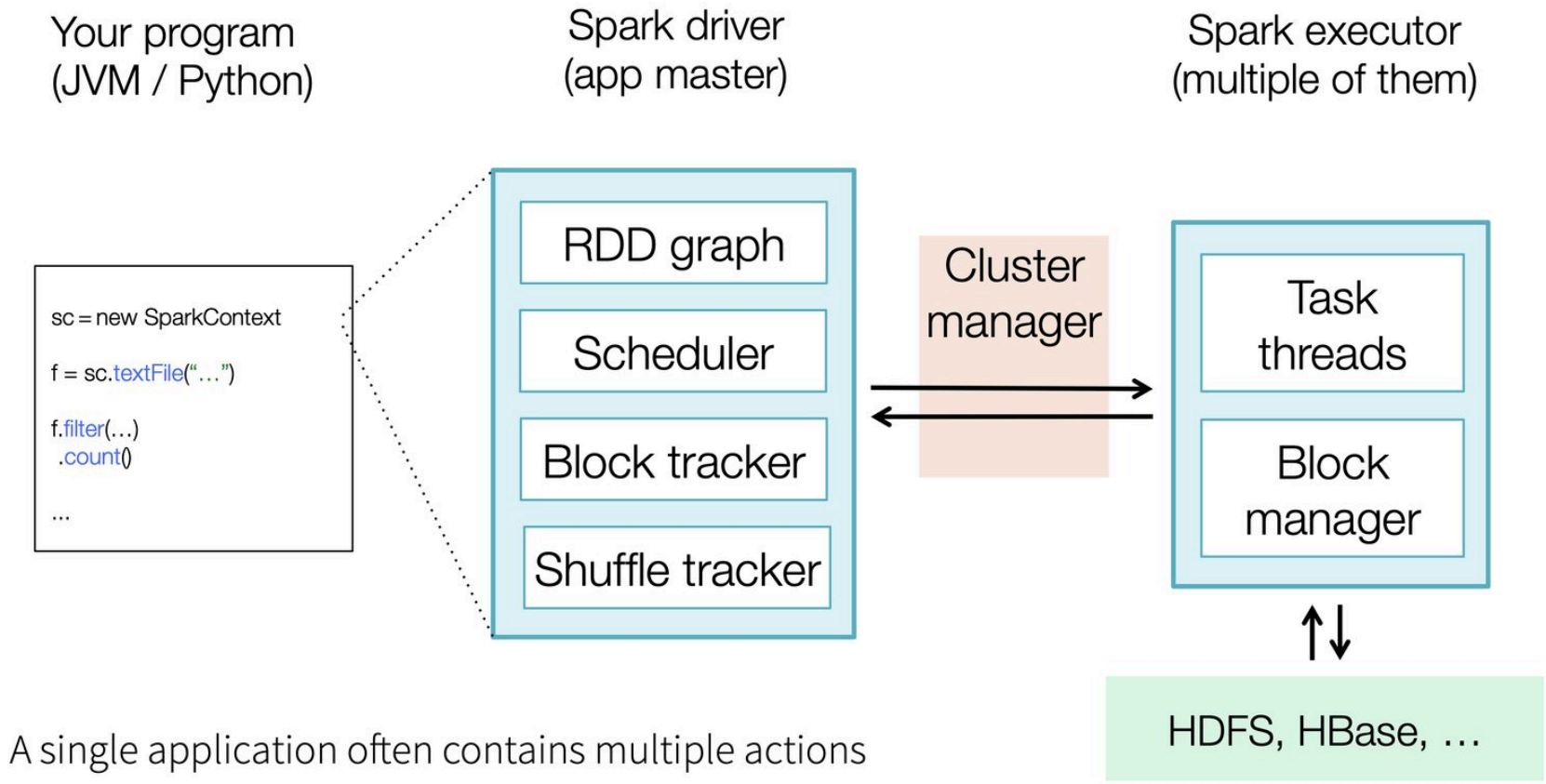
- Difficult to write more complex queries
  - Everything has to be expressed as map-reduce
- Need multiple MapReduce jobs: dramatically slows down because it writes all (intermediate) results to disk

Spark APIs



One core, four libraries, one DataFrame API on top

Driver and Executors



Driver plans, cluster manager hands out, **executors run and hold the blocks**

Why Spark? – Programmability

WordCount in 50+ lines of Java MR

```
public class WordCount {
  public static class TokenizerMapper
       extends Mapper<Object, Text, Text,
IntWritable> { ... }
  public static class IntSumReducer
       extends Reducer<Text, IntWritable,
Text, IntWritable> { ... }
  public static void main(String[] args) {
    Job job = Job.getInstance(conf, "word
count");

job.setMapperClass(TokenizerMapper.class);
...
  }
}
```

WordCount in 3 lines of Spark

```
val f = sc.textFile(inputPath)
val w = f.flatMap(l => l.split("
")).map(word => (word, 1)).cache()
w.reduceByKey(_ +
_).saveAsTextFile(outputPath)
```

RDDs: Partitions and Lineage

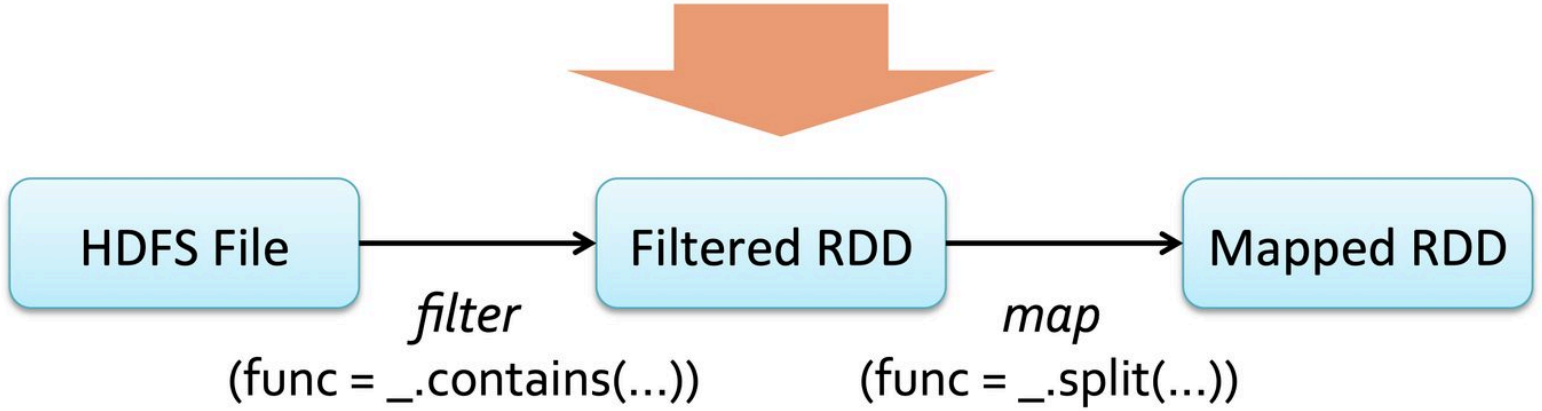
- RDD = Resilient Distributed Datasets
  - A partitioned, immutable collection with *lineage*
  - Lineage: expression that says how that relation was computed (e.g., a relational algebra plan)
- Transformations return RDDs; actions return values or write output
- If an executor loses partitions, the driver uses lineage to recompute them from available ancestors or source data

RDD Lineage

RDDs track *lineage* information that can be used to efficiently reconstruct lost partitions

Ex:

```
messages = textFile(...).filter(_.startsWith("ERROR"))
                        .map(_.split('\t')(2))
```



Each RDD remembers **the function that made it** from its parent

MapReduce vs RDD

Hadoop MapReduce: materialize between jobs
Spark: pipeline transformations and optionally cache reused data

Hadoop:

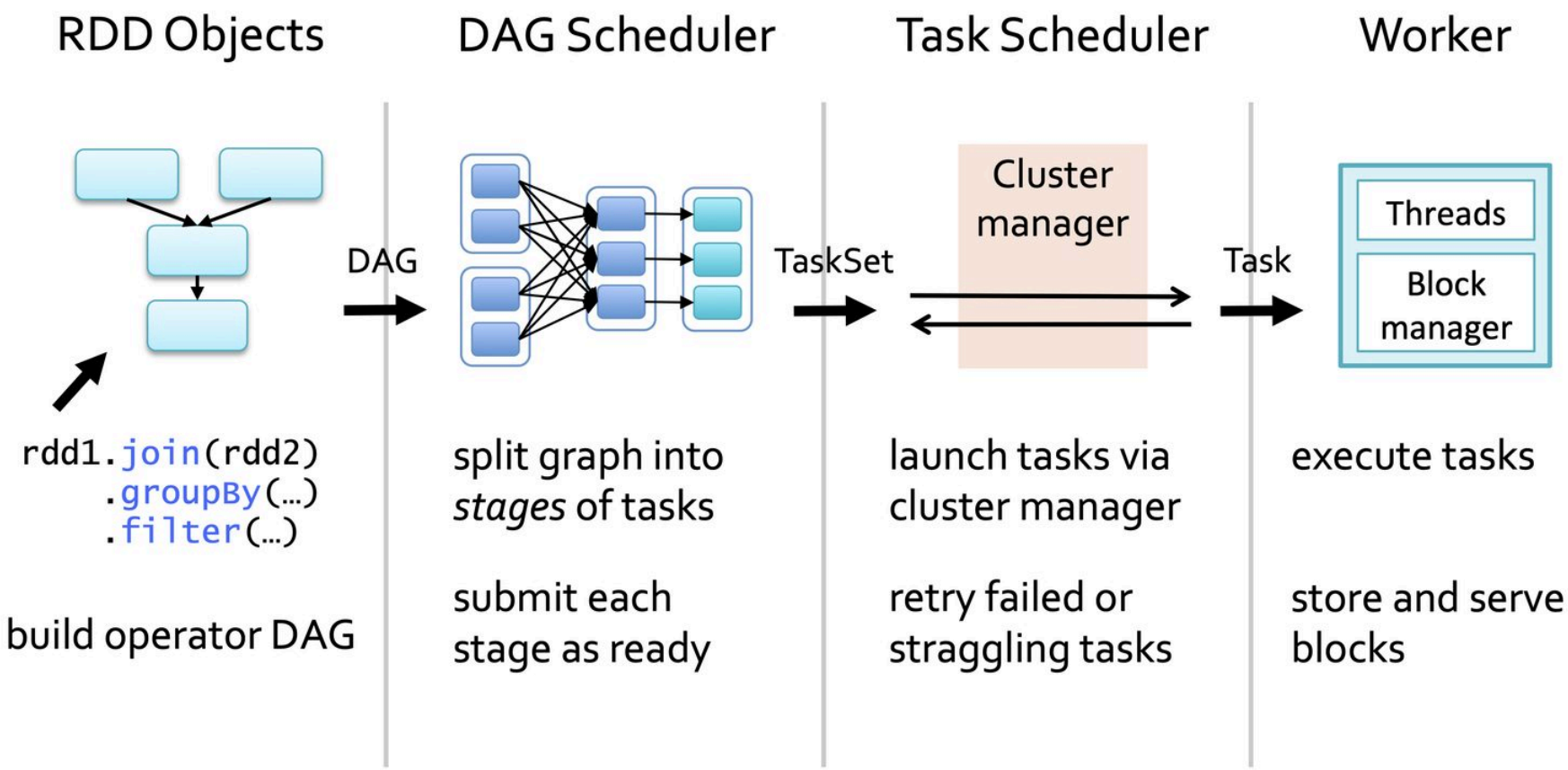
```
Map -> local disk -> shuffle
-> Reduce -> HDFS -> next job
```

Spark:

```
rdd.map(...).filter(...).groupBy(...).collect()
```

→ Spark builds a **DAG** internally; shuffles and spills can still use disk.

RDD Execution Process

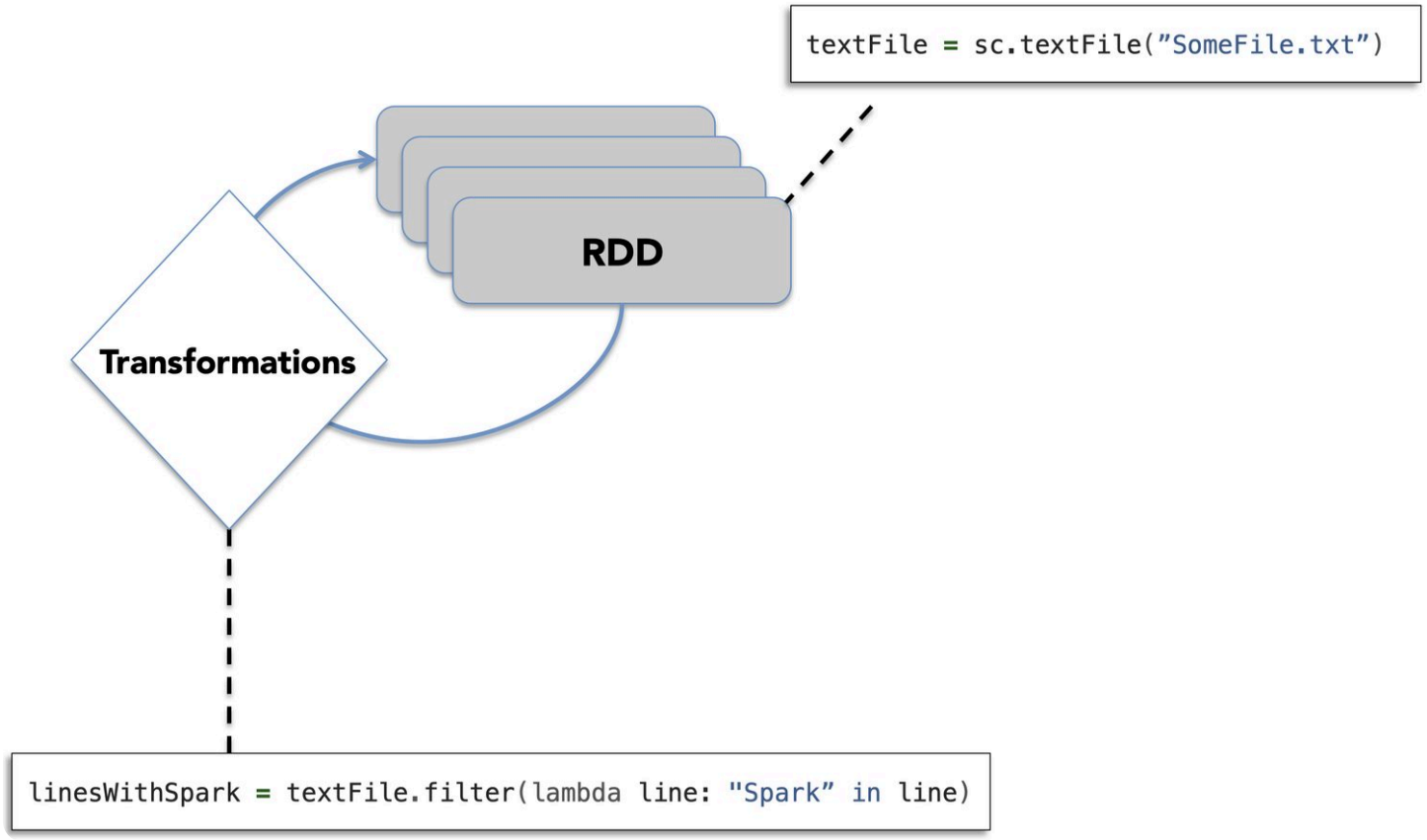


Code → DAG → stages → tasks → workers

Lazy Transformations and Actions

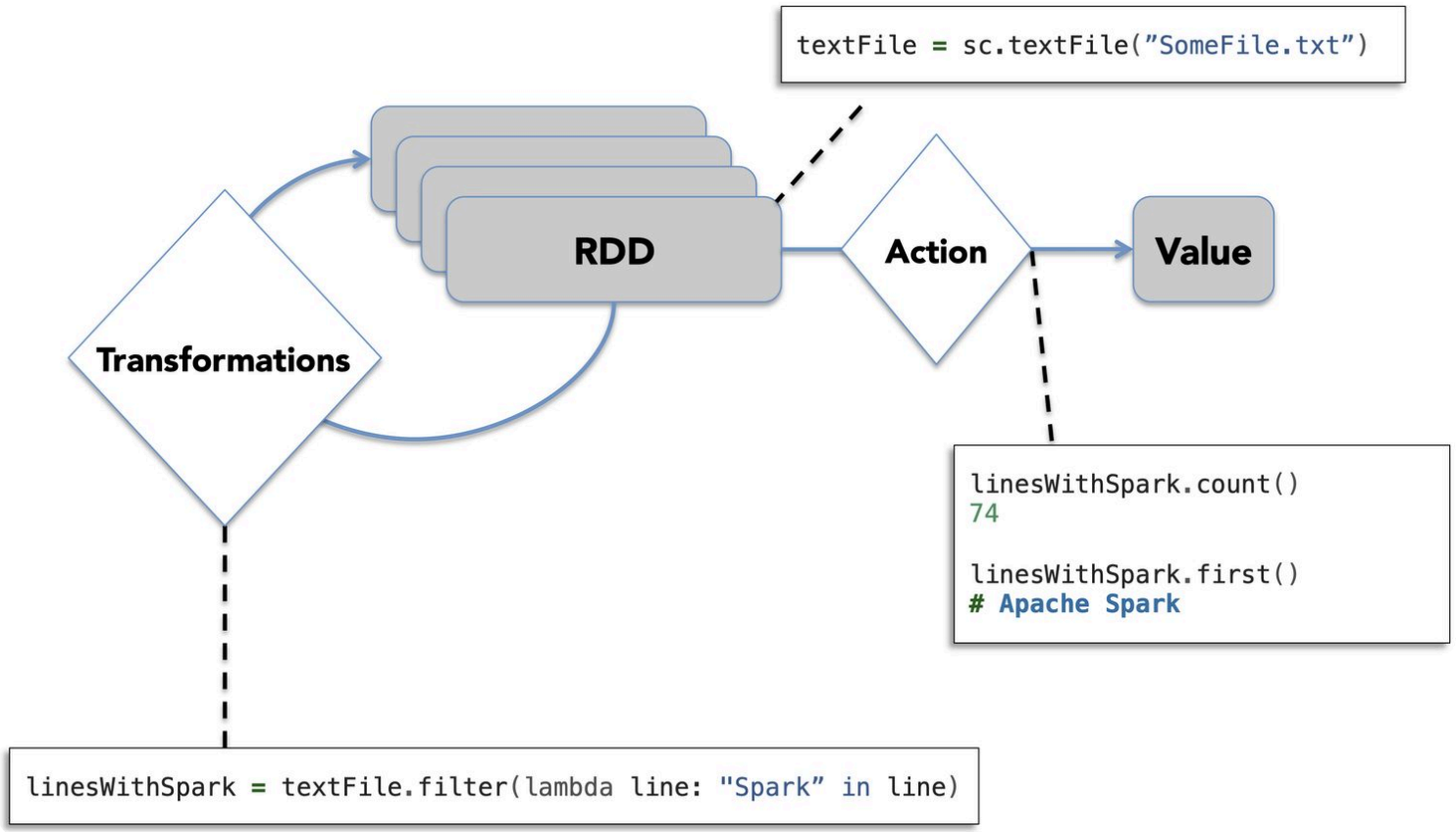
- A Spark program consists of:
  - Transformations (map, reduceByKey, join...). **Lazy**
  - Actions (count, reduce, save...). **Eager**
- **Eager**: operators are executed immediately
- **Lazy**: operators are not executed immediately
  - A *operator tree* is constructed in memory instead
  - Similar to a relational algebra tree

RDD Transformations



Transformations go from RDD to RDD; **nothing has run yet**

RDD Actions



An action turns the tree into a value; **that is when work happens**

RDD Programming – Persistence (a.k.a. Cache)

```
errors = sc.textFile("logs/").filter(
    lambda line: line.startswith("ERROR")).persist()

errors.count()  # first action computes and caches
errors.filter(lambda line: "sqlite" in line).count()
```

- Cached partitions can serve later actions
- Lost or evicted partitions are recomputed from available ancestors
- Caching is not a durable backup

persist() enables reuse; lineage enables recomputation

RDD Programming – Example

R(A,B)
S(A,C)

```
SELECT count(*) FROM R, S
WHERE  R.B > 200 AND S.C < 100 AND R.A = S.A
```

```
def pair(line):
    a, value = line.split(",")
    return int(a), int(value)
R = sc.textFile("R.csv").map(pair)
S = sc.textFile("S.csv").map(pair)
```

Parse headerless CSV lines into **(key, value)** pairs; join uses the first element as the key.

DataFrame Abstraction

Distributed collection of data grouped into named columns (i.e. RDD with schema)

- DSL designed for common tasks
  - Metadata
  - Sampling
  - Project, filter, aggregation, join, ...
  - UDFs

Available in Python, Scala, Java, and R (via SparkR)

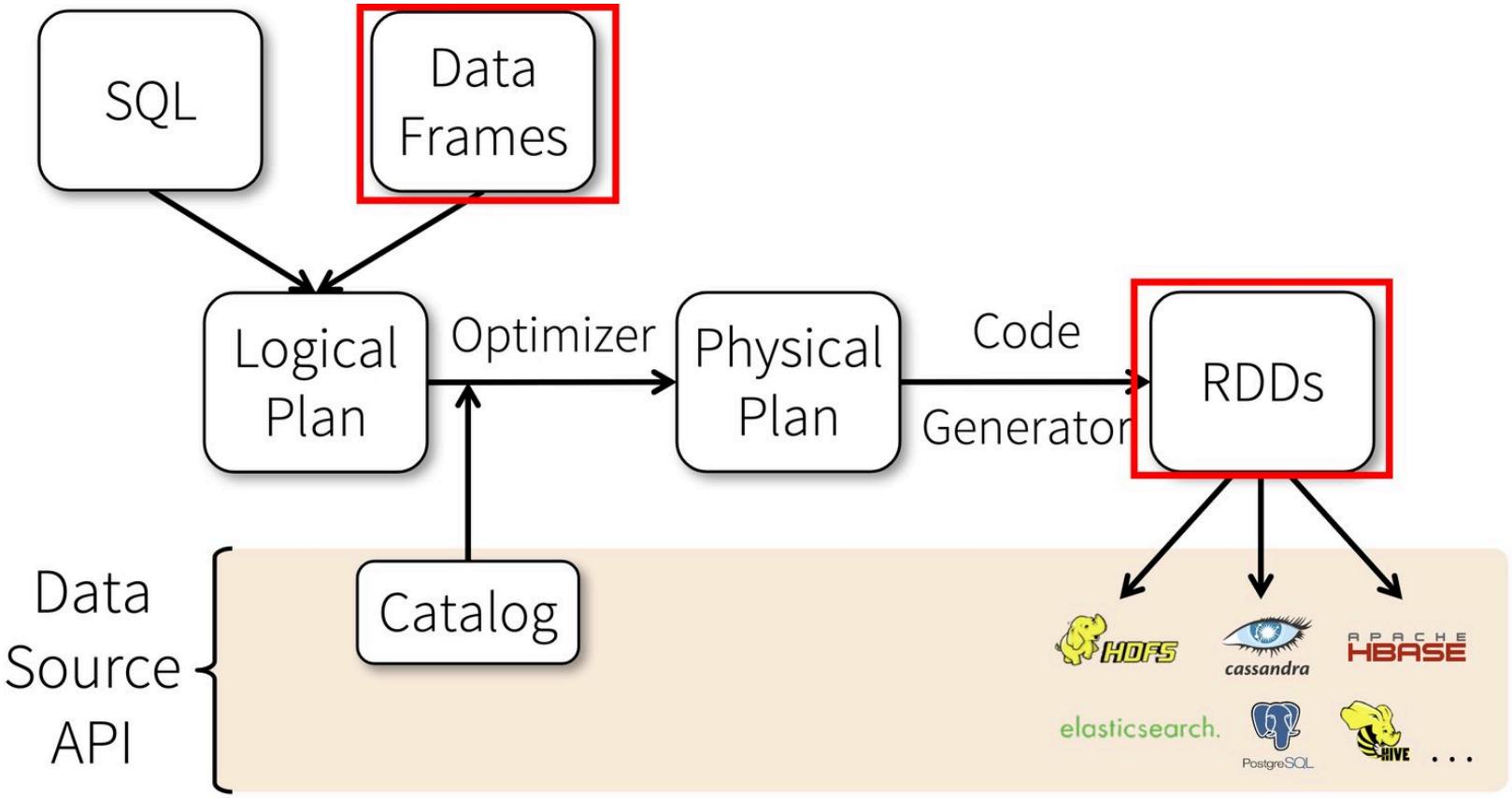
DataFrame Operations

DataFrames hold rows with a known schema and offer relational operations on them through a DSL

```
val users = spark.sql("select * from users")
val massUsers = users('country === "Canada")   // expression AST, not a closure
massUsers.count()
massUsers.groupBy("name").avg("age")
```

`'country === "Canada"` builds an **expression tree** Spark can read; a lambda in `filter` is opaque to the engine.

The Optimized Plan



SQL and DataFrames meet in a logical plan; **the optimizer is back**

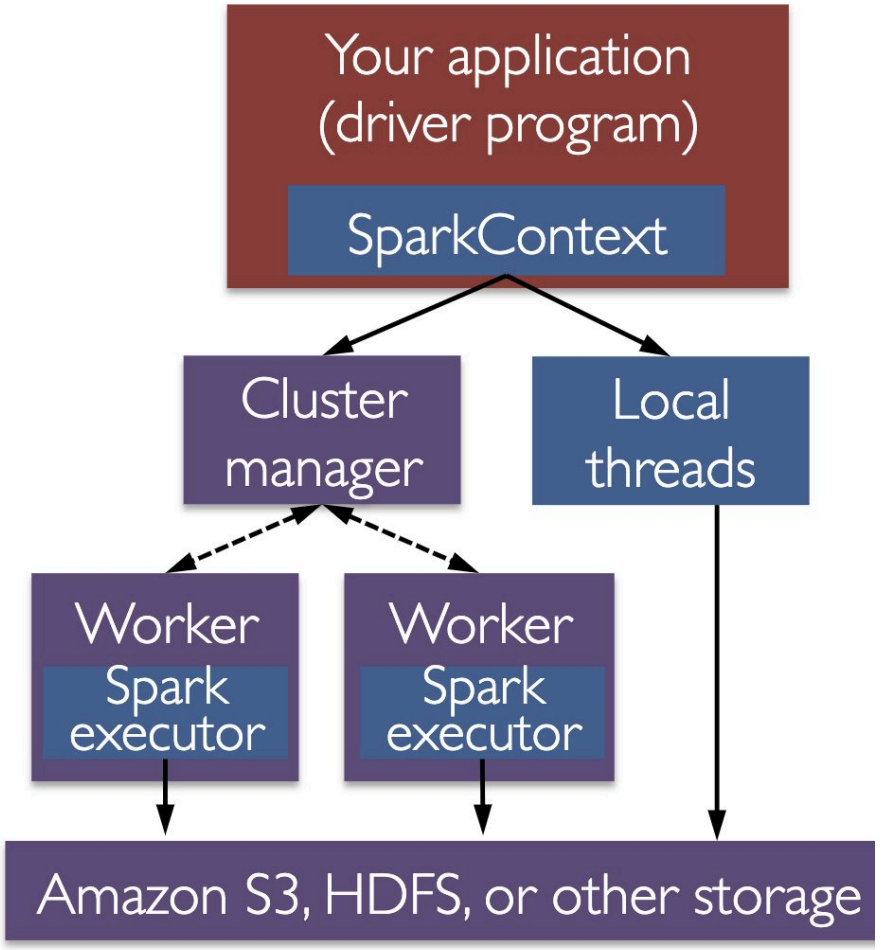
RDD vs DataFrame

Aspect	RDD	DataFrame
Level	Low-level API (manual control)	High-level API (table abstraction)
Optimization	No SQL expression optimizer	Catalyst optimizer (automatic)
Execution	User functions over objects	Optimized SQL expressions and generated code
Schema	Unstructured	Schema-aware
Ease of use	Complex functional code	Simple SQL-like syntax
When to use	Custom logic, unstructured data	ETL, analytics, ML pipelines

pySpark

- We are using the Python programming interface to Spark (pySpark)
- pySpark provides an easy-to-use programming abstraction and parallel runtime:
  - “Here’s an operation, run it on all of the data”
- `spark.apache.org/docs/latest/programming-guide.html`
- `spark.apache.org/docs/latest/api/python/index.html`

pySpark – Spark Driver and Workers



- A Spark program is two programs:
  - A driver program and a workers program
- Worker programs run on cluster nodes or in local threads
- RDDs are distributed across workers

pySpark – Creating RDDs

Create RDDs from Python collections (lists)

```
>>> data = [1, 2, 3, 4, 5]
>>> data
[1, 2, 3, 4, 5]

>>> rDD = sc.parallelize(data, 4)
>>> rDD
ParallelCollectionRDD[0] at
parallelize at PythonRDD.scala:229
```

`sc.parallelize()` creates a four-partition RDD; transformations wait for an action

Map and Filter in PySpark

```
>>> rdd = sc.parallelize([1, 2, 3, 4])
>>> rdd.map(lambda x: x * 2).collect()
[2, 4, 6, 8]

>>> rdd.filter(lambda x: x % 2 == 0).collect()
[2, 4]

>>> rdd2 = sc.parallelize([1, 4, 2, 2, 3])
>>> sorted(rdd2.distinct().collect())
[1, 2, 3, 4]
```

Function literals (lambdas) are closures automatically passed to workers

Key Aggregation in PySpark

```
>>> rdd = sc.parallelize([(1,2), (3,4), (3,6)])
>>> sorted(rdd.reduceByKey(lambda a, b: a + b).collect())
[(1, 2), (3, 10)]

>>> rdd2 = sc.parallelize([(1,'a'), (2,'c'), (1,'b')])
>>> rdd2.sortByKey().keys().collect()
[1, 1, 2]
```

pySpark – RDD Actions

```
>>> rdd = sc.parallelize([1, 2, 3])
>>> rdd.reduce(lambda a, b: a * b)
Value: 6

>>> rdd.take(2)
Value: [1,2] # as list

>>> rdd.collect()
Value: [1,2,3] # as list
```

DataFrames and pandas

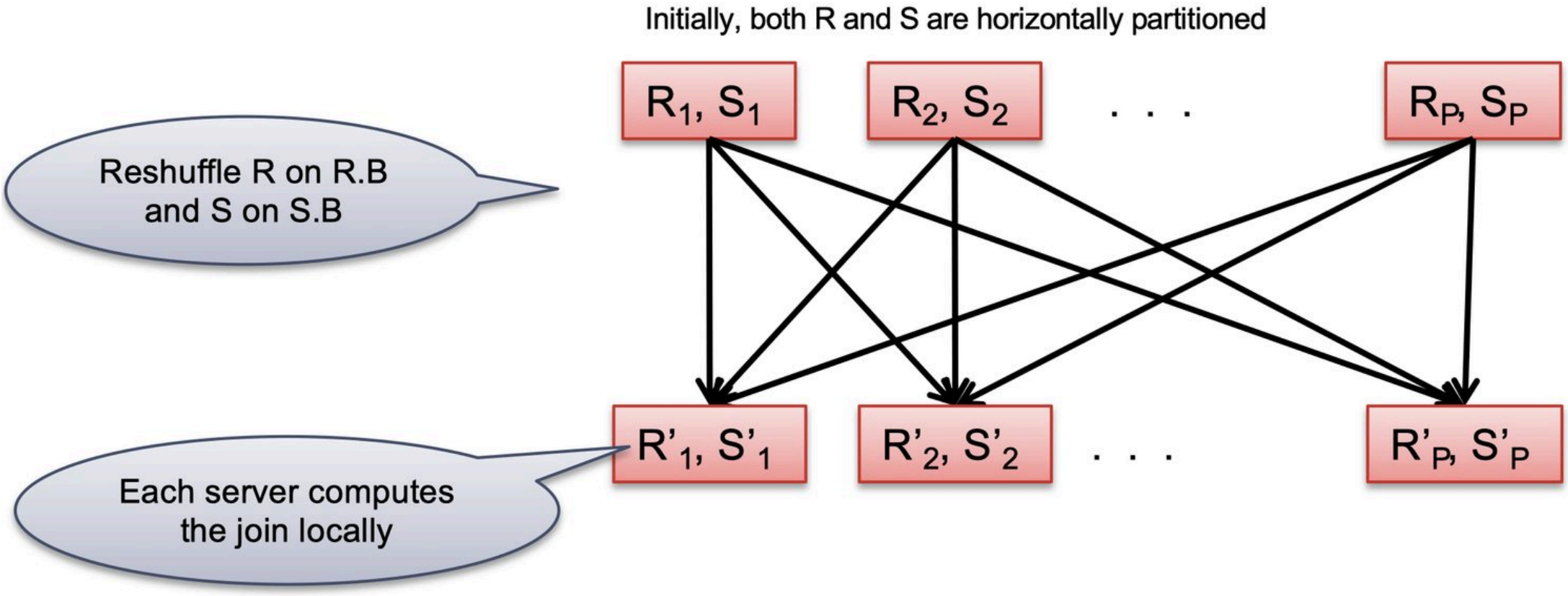
```
# Small results only: collect to the driver
pandas_df = spark_df.toPandas()

# Create a Spark DataFrame from Pandas
spark_df = spark.createDataFrame(pandas_df)
```

DataFrame Joins

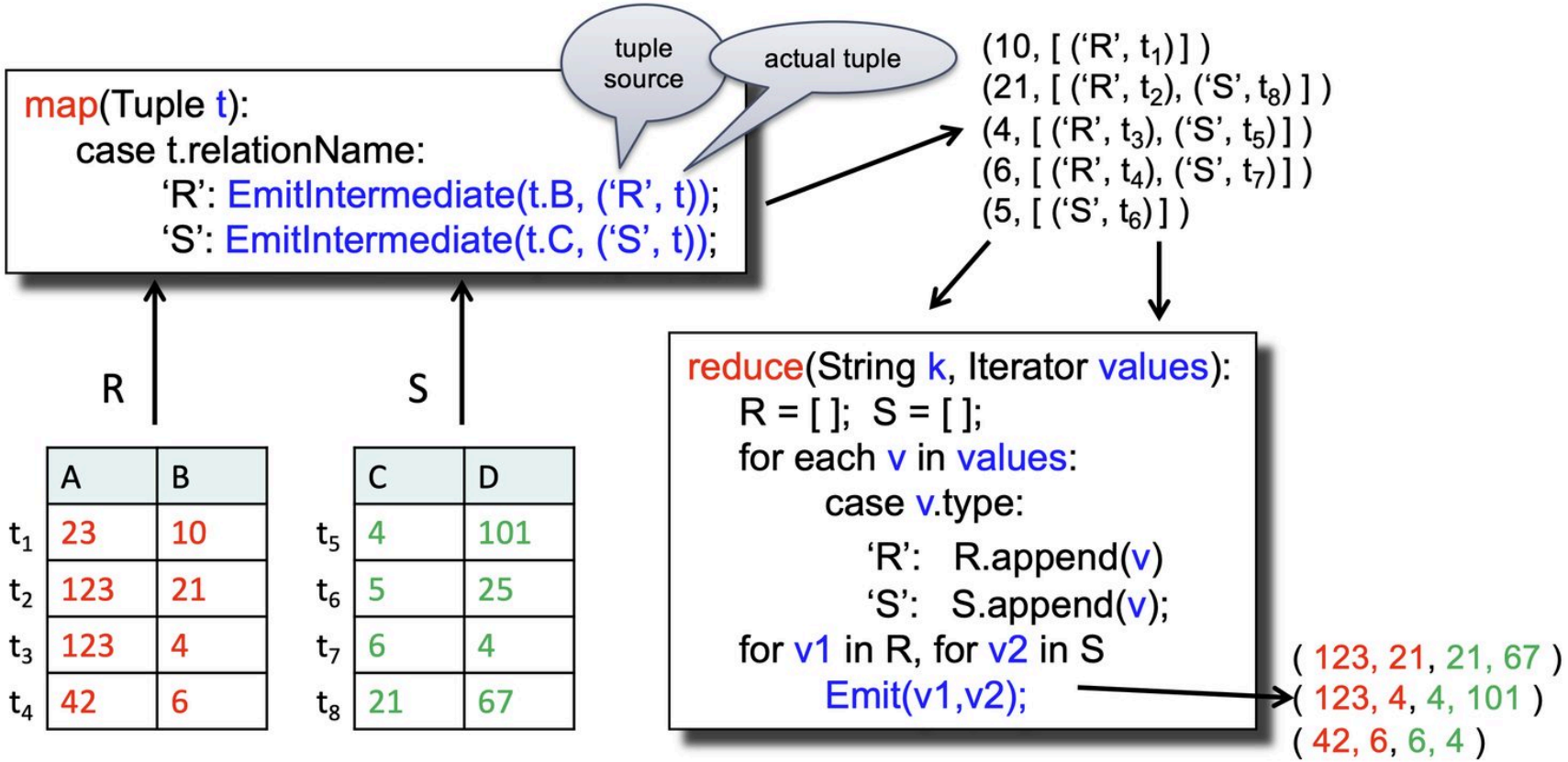
- SparkSQL and Spark DataFrames `join()` supports:
  - inner, outer, left outer, right outer, semijoin
- For Pair RDDs, pySpark supports:
  - inner `join()`, `leftOuterJoin()`, `rightOuterJoin()`, `fullOuterJoin()`

Shuffle Join



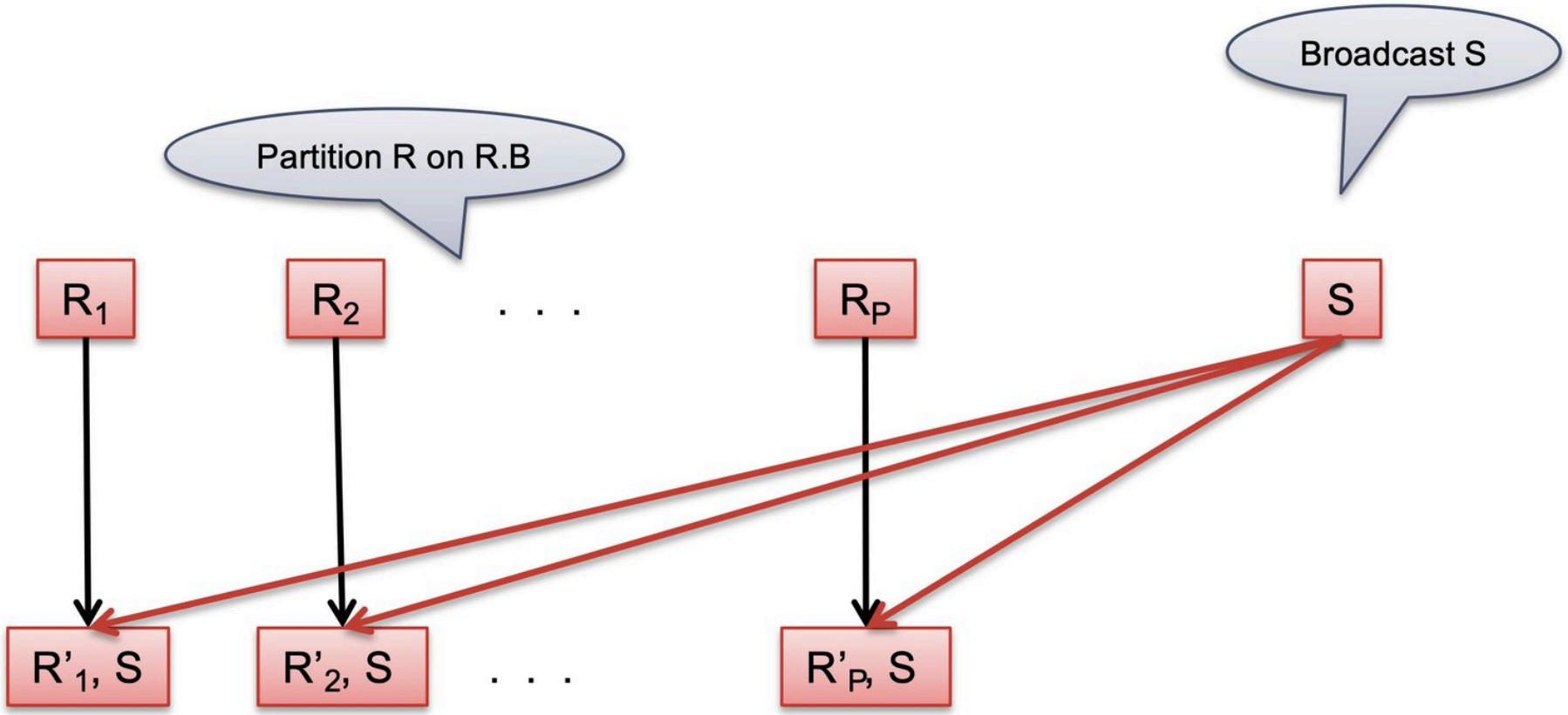
Reshuffle both sides on the join key, then **each server joins locally**

Shuffle Join by Key



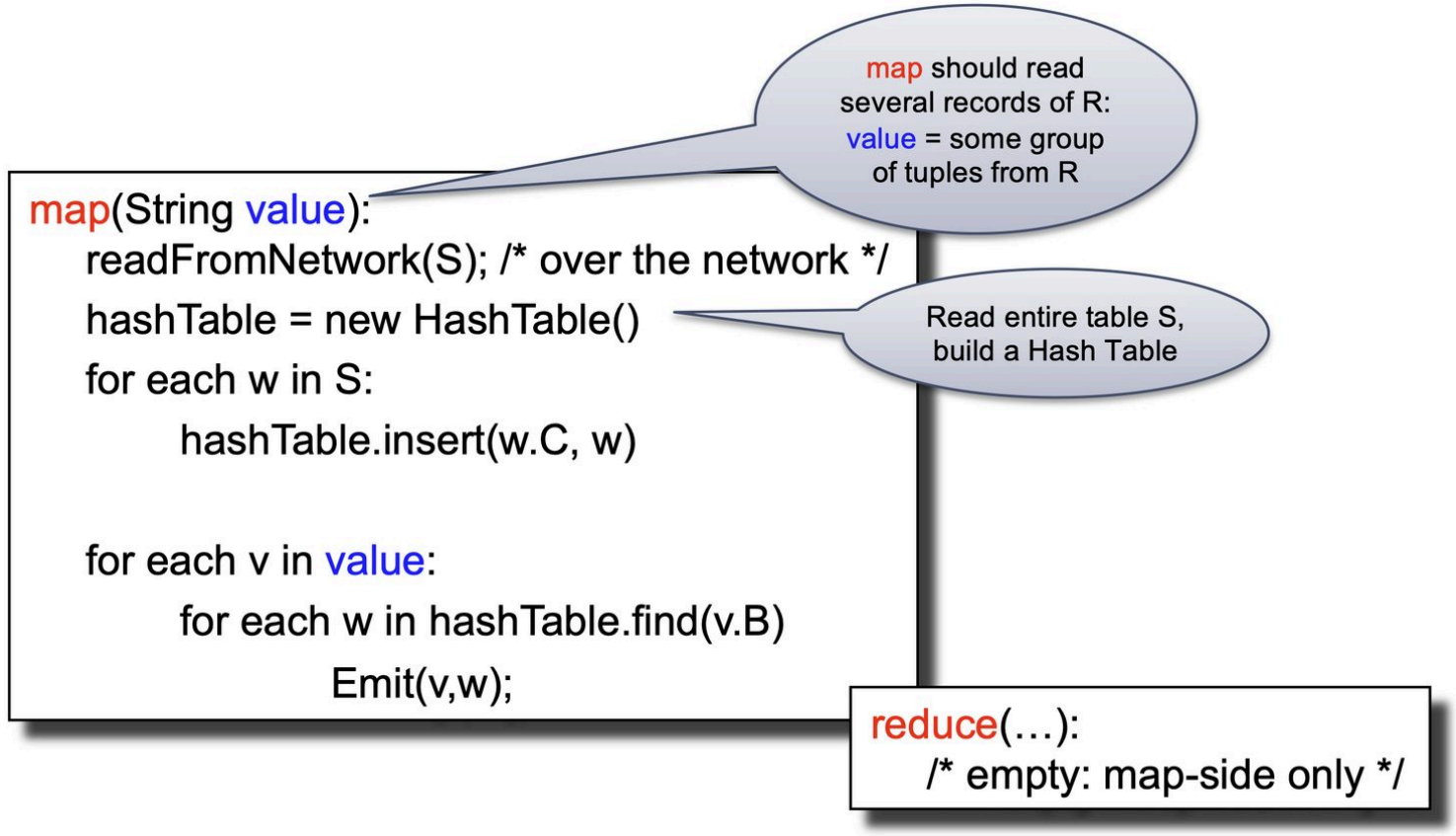
Tag each tuple with its source; the reduce **pairs R with S inside one key**

Broadcast Join



Broadcast the **small** table; the large input can keep its existing partitions

Broadcast Join in Each Worker



A hash table of S in every mapper; **no shuffle, no reduce**

Comparison of Join Algorithms

	Partitioned Hash Join	Broadcast Join
When to use	Both tables are large, or not already partitioned by the join key	One table is very small (MBs to tens of MBs) and can be replicated to all workers
Data scale	Large ↔ Large (fact–fact join)	Large ↔ Small (fact–dimension join)
Data movement	Both tables are shuffled across the network	Only the small table is sent to all nodes
Performance	Shuffle cost depends on size, partitioning and skew	Avoids shuffling the large input; broadcast and memory costs remain
Limitations	High shuffle cost, longer execution time	Small-table size limit, risk of out-of-memory
Rule of thumb	If both are big, partition	If one is small, broadcast

Spark SQL: One Query, Two Ways

```python
from pyspark.sql import functions as F

sales = spark.read.parquet("sales/")
(sales.filter(sales.region == "Asia")
      .groupBy("product")
      .agg(F.sum("amount").alias("total"))
      .orderBy(F.desc("total")))
```

```python
sales.createOrReplaceTempView("sales")
spark.sql("""
  SELECT product, SUM(amount) AS total
  FROM sales WHERE region = 'Asia'
  GROUP BY product ORDER BY total DESC""")
```

Both go through the **same optimizer** and run as the same plan

Files In, Files Out

```
events = (spark.read
          .option("header", True)
          .option("inferSchema", True)
          .csv("raw/events/*.csv"))

daily = events.groupBy("date", "machine").count()

(daily.write
       .mode("overwrite")
       .partitionBy("date")
       .parquet("clean/daily_counts/"))
```

Read raw files, transform, write **Parquet**; the write is the action that runs it all

Where Spark Runs

Where	Examples	Use it when
Your laptop	pip install pyspark, master local[*]	learning, testing on a sample
Managed platform	Databricks, AWS EMR, Google Dataproc	the data does not fit on one machine
Not Spark at all	pandas, Polars, DuckDB	the data fits on one machine

If the data **fits on one machine**, you do not need Spark

Takeaway: Big Data I

1. When data outgrows one machine, **map → shuffle → reduce** divides the work; a hot key can still leave one slow task
2. Spark uses **lineage** to recover lost partitions and can cache reused data
3. Write **DataFrames or SQL**; an action runs the lazy plan, and joins often pay for shuffles